git-flow cheatsheet

created by Daniel Kummer

efficient branching using git-flow by Vincent Driessen

translations: English - Castellano - Português Brasileiro - 繁體中文(Traditional Chinese) - 簡体中文(Simplified Chinese) - 日本語 - Türkçe - 한국어(Korean) - Français - Magyar(Hungarian) - Italiano - Nederlands - Русский (Russian) - Deutsch (German) - Català (Catalan) - Română (Romanian) - Ελληνικά (Greek) - Українська (Ukrainian) - Tiê´ng Việt (Vietnamese) - Polski - فارسی - العربية - Lietuviškai (Lithuanian) - Azərbaycanca (Azerbaijani) Bahasa Indonesia

About

git-flow are a set of git extensions to provide high-level repository operations for Vincent Driessen's branching model.

more

★★★

This cheatsheet shows the basic usage and effect of git-flow operations

★★★

Basic tips

★ Git flow provides excellent command line help and output. Read it carefully to see what's happening...
★ The macOS/Windows Client *Sourcetree* is an excellent git gui and provides git-flow support
★ Git-flow is a merge based solution. It doesn't rebase feature branches.

★★★

Setup

★ You need a working git installation as prerequisite.
★ Git flow works on macOS, Linux and Windows

★★★

macOS

Homebrew

```
$ brew install git-flow-avh
```

Macports

```
$ port install git-flow-avh
```

Linux

```
$ apt-get install git-flow
```

Windows (Cygwin)

For detailed git flow installation instructions please visit the *git flow wiki*.

```
$ wget -q -O - --no-check-certificate https://raw.github.com/petervanderdoes/gitflow-avh/develop/contrib/gitflow-installer.sh install stable | bash
```

You need wget and util-linux to install git-flow.

Getting started

Git flow needs to be initialized in order to customize your project setup.

★★★

Initialize

Start using git-flow by initializing it inside an existing git repository:

```
git flow init
```

**You'll have to answer a few questions regarding the naming conventions for your branches.
It's recommended to use the default values.**

Features

★ Develop new features for upcoming releases
★ Typically exist in developers repos only

★★★

Start a new feature

Development of new features starting from the 'develop' branch.

Start developing a new feature with

```
git flow feature start MYFEATURE
```

This action creates a new feature branch based on 'develop' and switches to it

Finish up a feature

Finish the development of a feature. This action performs the following

★ Merges MYFEATURE into 'develop'
★ Removes the feature branch
★ Switches back to 'develop' branch

```
git flow feature finish MYFEATURE
```

Publish a feature

Are you developing a feature in collaboration?
Publish a feature to the remote server so it can be used by other users.

```
git flow feature publish MYFEATURE
```

Getting a published feature

Get a feature published by another user.

```
git flow feature pull origin MYFEATURE
```

You can track a feature on origin by using

```
git flow feature track MYFEATURE
```

Make a release

★ Support preparation of a new production release
★ Allow for minor bug fixes and preparing meta-data for a release

★★★

Start a release

To start a release, use the git flow release command. It creates a release branch created from the 'develop' branch.

```
git flow release start RELEASE [BASE]
```

You can optionally supply a `[BASE]` commit sha-1 hash to start the release from. The commit must be on the 'develop' branch.

★★★

It's wise to publish the release branch after creating it to allow release commits by other developers. Do it similar to feature publishing with the command:

```
git flow release publish RELEASE
```

(You can track a remote release with the `git flow release track RELEASE` command)

Finish up a release

Finishing a release is one of the big steps in git branching. It performs several actions:

★ Merges the release branch back into 'master'

★ Tags the release with its name
★ Back-merges the release into 'develop'
★ Removes the release branch

```
git flow release finish RELEASE
```

Don't forget to push your tags with `git push origin --tags`

Hotfixes

★ Hotfixes arise from the necessity to act immediately upon an undesired state of a live production version
★ May be branched off from the corresponding tag on the master branch that marks the production version.

★★★

git flow hotfix start

Like the other git flow commands, a hotfix is started with

```
git flow hotfix start VERSION [BASENAME]
```

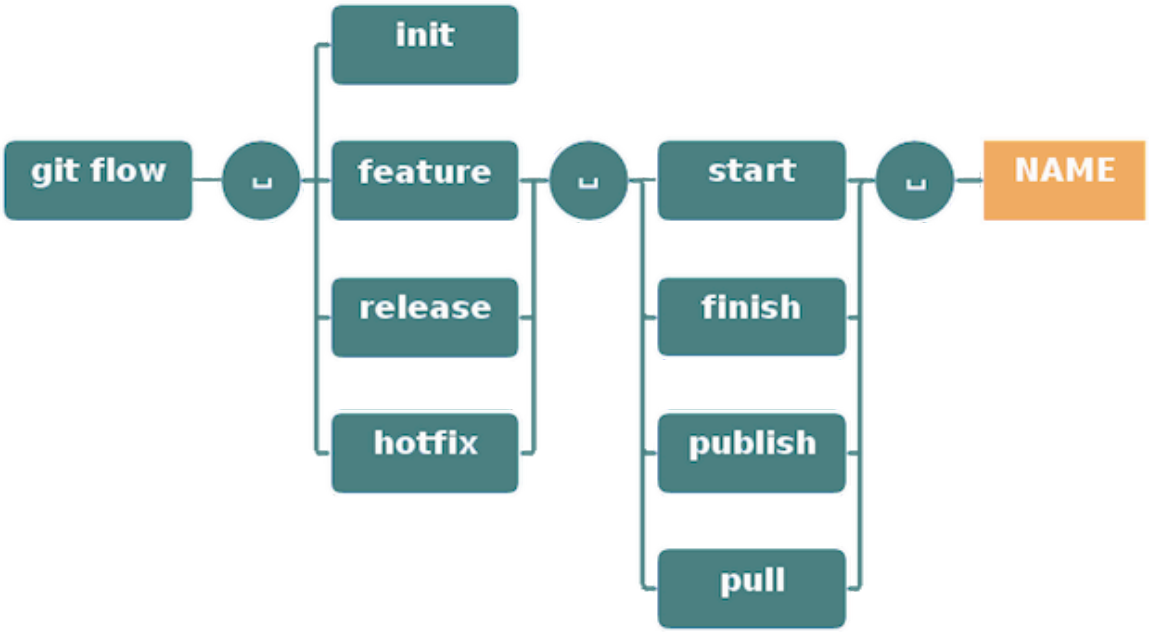
The version argument hereby marks the new hotfix release name. Optionally you can specify a basename to start from.

Finish a hotfix

By finishing a hotfix it gets merged back into develop and master. Additionally the master merge is tagged with the hotfix version.

```
git flow hotfix finish
VERSION
```

Commands



Backlog

★★★

★ Not all available commands are covered here, only the most important ones

★ You can still use git and all its commands normally as you know them, git flow is only a tooling collection
★ The 'support' feature is still beta, using it is not advised
★ If you'd like to supply translations I'd be happy to integrate them

★★★

Comments

B **Ben Yitzhaki**
9 years ago

what is the best practice for fixing issues in release sprint while a new release is already on development (so we can't edit the develop branch)? do them directly on the release branch? how can we handle Pull Request for the fixes done on the release branch etc..?

4 0 Reply Share ›

E **Evan Butler** → Ben Yitzhaki
8 years ago

IMO yes, make a branch off of releaseCandidate, then just create a PR to merge the fix branch into releaseCandidate. Then the fix goes out with that release. After the release is merged with master, you can backmerge master into develop to get those changes into develop.

1 0 Reply Share ›

H **HK** → Ben Yitzhaki
9 years ago

in git flow the develop branch contains the code for the next release. if this code needs bugfixes then you are not ready for the release and should continue with the development version of the code.

0 0 Reply Share ›

Ben Yitzhaki → HK
9 years ago

Thats not practical. In real life, you have bugs on the release while you got to continue working on new features and eventually merge them to develop, even if thete are still open issues on the previous release

10 0 Reply Share ›

Tim Anderson → Ben Yitzhaki
8 years ago

You should still run a hotfix that fixes the code in both the master and the develop branch. All features should be rebasing before finishing anyway, so this shouldn't be an issue.

1 1 Reply Share ›

J **Julien Rougeron** ➚ Ben Yitzhaki

9 years ago

I would say you open an hotfix on the master branch to fix the bug that came out when releasing

0 1 Reply Share ›

 **Pablo Ezequiel Leone Signetti** ➚ Julien Rougeron

7 years ago

You must never release any code/version with issues... I agree 100% with Tim! It's a terrible practice to release code with an error to then deploy a hotfix and remember that each deployment process could put down the app for some time and this is just one of the several issues you'll face if you do what Julien recommends.

1 1 Reply Share ›

 **Rémi Alvergnat** ➚ Pablo Ezequiel Leone Signetti

6 years ago

Welcome to real world :)

2 0 Reply Share ›

R **Ren Lawrence**

9 years ago

What do you suggest is the best practice when there are commits in develop that are not ready for the release when it is cut (maybe it's a feature client is not trained for yet)? I suppose the answer is: cut the release earlier, but that's not always practical. I've been in the habit of either adding revert commits to the release branch or cherry-picking commits into the release branch, but that hardly feels like best practice.

3 0 Reply Share ›

 **danielkummer** Mod ➚ Ren Lawrence

9 years ago

I'm not sure I understand your question correctly - but if you're able to pick a specific commit in the develop branch you deem "production ready" you can simply checkout that specific commit via its SHA-1 hash and then branch away the current release branch from there. If you need another specific commit further "up" the history for that specific release doing a cherry pick is a valid approach in my opinion as it's a lot easier and less error prone than creating a patch file...

0 0 Reply Share ›

R **Ren Lawrence** ➚ danielkummer

9 years ago

The problem is, that the "production ready" features might be mixed up chronologically with "non-production ready" features, so you can't just reset the head in that case.

To avoid cherry-picking/reverting in the future I think the solution seems to be a.) cut branch early and b.) add feature flags for features that aren't ready yet.

2 0 Reply Share ›

danielkummer Mod ↗ Ren Lawrence

9 years ago

I see your issue - in the future I'd recommend using more feature branches and merging them back on feature completeness...

2 0 Reply Share ›

S

Sidney de Moraes

9 years ago

"Commands" section misses the "track command", doesn't it?

3 0 Reply Share ›

danielkummer Mod ↗ Sidney de Moraes

9 years ago

You're right! However I currently don't have the time to adjust the graphics for it, I'm sorry about that.

0 0 Reply Share ›

S

Sidney de Moraes ↗ danielkummer

9 years ago

Can I help?

0 0 Reply Share ›

sonicyoof ↗ Sidney de Moraes

8 years ago

This page is on GH so you can open a PR: https://github.com/danielku...

0 0 Reply Share ›

Rizky Syazuli

9 years ago

question: should i deploy before or after finishing the release/hotfix? another thing, i want to have a build task that appends the version number to my js filenames. supposedly i can do it by using `git describe` command to get the latest tag. but if i haven't finish the release/hotfix, that command only gets the previous release/hotfix, and not the current one.

2 0 Reply Share ›

B

blocktrucks.io ↗ Rizky Syazuli

8 years ago

You should always merge your release or hotfix back into master first, then tag the release in master, then deploy from master. Think of master as your single point of reference for all versions of your software you have released.

1 0 Reply Share ›

FF

Free Free Palestine

2 years ago

Stop funding and supporting Israel genocide.

1 0 Reply Share ›

ImBIOS

3 years ago

Can we use squash n merge all over the place here? os Should we use merge commit?

It's clear it's not using rebase commit right.

1 0 Reply Share ›

Amorn Kingthong

4 years ago

Go

1 0 Reply Share ›

TheBoneJarmer

7 years ago

Great work for the cheatsheet! However, you are missing "git flow bugfix", which is very essential imho. When I Google for "how to git flow for bugs" I get articles suggesting to use feature branches because back in the time, the bugfix command did not exist.

1 0 Reply Share ›

Pablo Ezequiel Leone Signetti

7 years ago

For those who want something more practical...

Hotfixes are from master.
Bugfixes can be either from release/latest or development and tag the version that are fixing, current version on development or the next version in the release/latest branch.
You can bugfix a feature in development.
You can bugfix the release/latest branch before the official release.
Having a release/1.0.0 is useless in some environments, where preproduction is not generated automatically by CI. Our system is serving release/latest in preproduction and we always merge development into release/latest to be able to test the next release. Once is fully finished we create release/{version} to meet the standard and after merging into master we tag the new version.

As you can see we have, hotfix/xxx, feature/xxx, bugfix/xxx, release/xxx, development and master branches.

xxx: task code in Jira, Trello, etc...

Hope this helps!

1 0 Reply Share ›

Strange

8 months ago

Free palestine.

0 0 Reply Share ›

Andrew Hardin
2 years ago

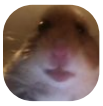
How do I merge develop branch without deleting my feature branch with git flow ?

o o Reply Share ›

Ryan Mbr
2 years ago

#FREE_PALESTINE

o o Reply Share ›



j23
2 years ago

hi

o o Reply Share ›

Rafael Corrêa Gomes
3 years ago

Melhor referência!

o o Reply Share ›

Mister X
4 years ago

how unninstall git flow of project?

o o Reply Share ›

Strange ↗ Mister X
8 months ago

git config --remove-section gitflow.branch
git config --remove-section gitflow.prefix
git config --remove-section gitflow.path

o o Reply Share ›

Renato Lucena
5 years ago

Otimo

o o Reply Share ›

Francisco
5 years ago

What's the naming convention for UI changes/updates? The feature is already done but now there's a UI change.

o o Reply Share ›

M **mehdi rochdi**
5 years ago

how i create branch like 1.x when finishing release

o o Reply Share ›

MR **medhi rochdi**
5 years ago

Hello,
how i can change branch master by version like 1.x, 2.x etc

o o Reply Share ›

A **Ahmet Ates**
5 years ago

Very helpful. Thanks for the info.

o o Reply Share ›

 **Abelardo León González**
6 years ago

Remember: first a pull, then create any branch and, finally, modify your code.

o o Reply Share ›

 **Dani**
7 years ago

re loco !

o o Reply Share ›

M **M**
7 years ago

is there (git flow pull) command ?

o o Reply Share ›

D **David**
7 years ago edited

First of all thank you for this very useful page!
I've got a question about the *Make a release* > *Finish up a release* paragraph.

Merging any release branch into develop should be quite straightforward each time, eg. without any conflict.

What bothers me is that according to the graph above, merging a release branch into master should be straightforward as well!
Indeed, the merge should work for release branch 1. However, for release branch 2 (and above), merge conflicts will certainly occur.
Why? Because commits being part of release branch 1, previously merged into master, will be merged again when merging release branch 2 into master!

So, my question is: should merging a release branch into master (at least from release branch 2) always be done with force overwrite?

o o Reply Share ›

HU **Haisum Usman**
7 years ago

Thanks for the information.

o o Reply Share ›

V **Vic**
7 years ago

Awesome information. Especially for people who is starting on Coding world.
Congratulations

o o Reply Share ›

 **Gabriel Maia**
7 years ago

I'm a newbie to git-flow and I want to implement it in a team that requires code reviewing every time under a pull request. How does pull requests are made using this type of workflow?

o o Reply Share ›

 **Rémi Alvergnat** → Gabriel Maia
6 years ago

Pull requests are created from feature branches and hotfixes branches before merging

o o Reply Share ›

L **lusajo malopa**
8 years ago

i keep getting this every time i try to create release branch 'git flow release start 1.1.5' fatal: '?[D?[D?[D?[D?[C?[C?[C1.1.5' is not a valid branch name.
Fatal: Could not create release branch 1.1.5'.

o o Reply Share ›

J **Jono Watkins** → lusajo malopa
2 years ago

I get his was six years ago, but i fixed it. gitconfig file

o o Reply Share ›

J **Jono Watkins** → lusajo malopa
2 years ago

i have the same issue, was a solution found at all?

o o Reply Share ›

 **Nirav**
8 years ago

Hello Daniel,

I am confused what to do in my scenario. We have a team with 15 to 20 developer.
We have 3 branches as of now. 1) Develop 2) Beta 3) Master

For new feature every one checkout with new local branch from 1)dev.
After finishing the feature developer will push the changes to its local and origin dev branch.

In between some other developer had also made the changes and pushed to dev branch with no conflicts.

But now i want to release the 1st developer feature changes to beta branch, and when i try to merge that changes it gives shows the all changes done by other developer as well in the selection.

So how to tackle with these scenario, i there any flow to release feature wise functionality from dev > to beta > to master.

Pls guide.

Thanks in advance.

0 0 Reply Share ›

Tim Anderson → Nirav

8 years ago

Your team should be running git flow feature rebase before finishing a feature to ensure this doesn't happen. Before finishing, run the following: git checkout develop && git pull && git flow feature checkout [your-feature] && git flow feature rebase. Then fix conflicts, git add, git rebase --continue, then git flow feature finish

1 0 Reply Share ›

Vytautas

8 years ago

What if I want to add multiplefixes to a hotfix release this forses me to create tag every time I fix a issue why cant this be optional?

0 0 Reply Share ›

Tim Anderson → Vytautas

8 years ago

Why is this an issue for you? This is the whole idea behind the workflow - to make it easy to understand what has happened and when, and to enforce semver rules.

0 0 Reply Share ›

Load more comments